Auxiliary Tasks to Improve Final Prediction

Evan Meltzer

University of Washington

1. Introduction:

Many deep learning tasks produce deceptively simple outputs while requiring models to reason through complex intermediate representations. My undergraduate research focuses on one such task: predicting whether a protein will bind to a small molecule. This problem is central to synthetic biology because accurate binding predictions can accelerate the discovery of vital de-novo proteins and compounds. For example, binding prediction can help identify functional drug candidates before expensive laboratory testing.

Although protein-ligand binding is a binary classification task, it depends on many complex biological factors, including protein structure, binding-pocket geometry, and local chemical interactions. To address this complexity, my lab develops models that make auxiliary predictions before producing a final binding prediction. My project investigates the efficacy of this approach by asking: how does stacking multiple auxiliary predictions before a final prediction impact model performance?

To study this question, I use image geolocation as a toy problem. Image geolocation is well suited for this purpose because its final output, a latitude-longitude pair, requires a model to infer many intermediate features, such as climate, terrain, country, and proximity to coastlines. In this project, I compare five architectures that incorporate prediction of these auxiliary features in different ways.

I expect auxiliary predictions to improve model performance for two reasons. First, they may encourage the model to learn denser feature representations. Second, they may break the difficult problem into more manageable subtasks. Additionally, I believe auxiliary predictions may improve interpretability because, rather than only producing a final output, the model can reveal intermediate predictions that indicate what evidence it is using.

2. Related Work

This section reviews prior work in two areas: protein-ligand modeling, which motivates the research problem, and image geolocation, which serves as this project's experimental domain.

In protein-ligand modeling, recent architectures have used multi-task learning to improve prediction. Hu et al. introduce Multi-PLI, a model that improves binary protein-ligand binding classification by simultaneously performing binding-affinity regression [3]. He et al. propose LigPose, which predicts the three-dimensional structure of protein-ligand complexes while using binding strength and atomic interactions as auxiliary tasks [2]. Yan et al. extend this idea to pretraining through Multi-task Bioassay Pre-training, a framework that treats different affinity measurements, such as IC50, Ki, and Kd, as separate pretraining tasks [5]. Together, these studies show how related biochemical supervision signals can be combined to support prediction.

Image geolocation has also been studied as a task which requires combining many contextual signals. Müller-Budack et al. formulate geolocation as classification over geographic cells and incorporate both hierarchical geographic partitions and scene information, such as whether an image is indoor, natural, or urban [4]. Haas et al. introduce PIGEON, an image geolocation system that combines geocell creation, multi-task contrastive pretraining, and a

geolocation-specific loss [1]. These works show that geolocation can benefit from auxiliary tasks which help models capture intermediate structure.

Our project builds on this literature by proposing multiple architectures that incorporate auxiliary predictions before making a final geolocation prediction

3. Data

3.1 Data Collection

Data for this project was sourced from the Open Street View 5M dataset. This dataset was selected because it contains a large collection of street-view images from diverse geographic regions, with each image annotated using geographic and environmental metadata. For this project, the labels used for each sample were latitude, longitude, country, climate, land cover, and distance to sea.

Figure 1: Example training image and corresponding labels.

The training data was drawn from the training split of the OSV 5M dataset. The full training split contains 97 folders of images. Due to compute limitations, this project used only the first four folders, corresponding to train/00.zip through train/03.zip on Hugging Face. Before curation, this subset contained 200,000 samples.

Evaluation and test data were drawn from the test split. The full test split contains five folders, of which only the first folder, test/00.zip, was used. This folder contained approximately 50,000 samples. These samples were randomly divided into equal evaluation and test sets, producing 25,000 samples in each split before curation.

3.2 Data Curation

The dataset was highly imbalanced across geographic regions and environmental categories. This created two issues. First, some countries, climates, and land-cover classes appeared very rarely, making them difficult for the model to learn. Second, some classes appeared in the eval or test sets but were absent from the training set, making it impossible for the model to predict those labels.

To address this, we filtered the dataset based on label frequency in the training set. For each categorical label type, we counted the number of training samples associated with each class. Samples were retained only if their country, climate, and land-cover labels belonged to classes that appeared at least a threshold number of times in the training set. This ensured that all retained categorical labels had sufficient representation during training and eliminated unseen classes in evaluation and testing.

Figure 2: Countries represented in the curated train set.

After curation, the training set was reduced from approximately 200,000 samples to 170,530 samples. The evaluation and test sets were each reduced from 25,000 samples to 15,325 samples.

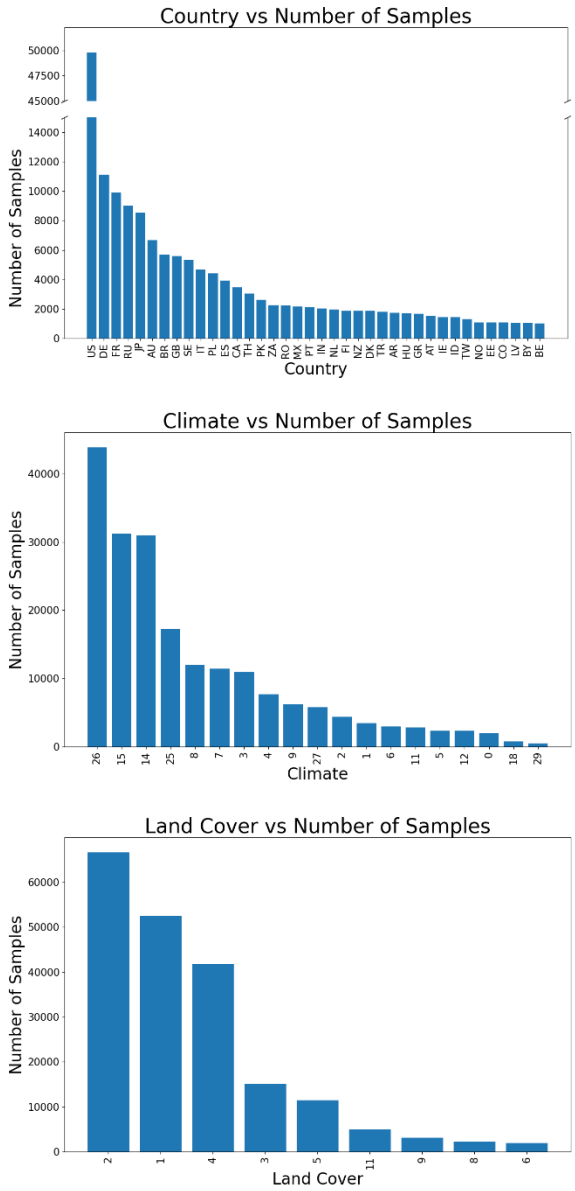


Figure 3: Top: Distribution of country labels across curated samples. Samples were retained only if more than 1,000 train samples shared the same country label. Middle: Distribution of climate labels across curated samples. Samples were retained only if more than 200 train samples shared the same country label. Bottom: Distribution of land cover labels across curated samples. Samples were retained only if more than 2,000 train samples shared the same land cover label.

3.3 Data Preprocessing

Images in the dataset had varying spatial dimensions. To support batching, all images were standardized to the most common resolution, 512 pixels in height by 910 pixels in width. Images larger than the target size were center-cropped, while images smaller than the target size were resized through reflective padding. After spatial standardization, the RGB channels were normalized independently using the mean and standard deviation computed from the training set.

4. Experiments

4.1 Overview

We conducted five experiments to investigate the research question. Each experiment used a distinct model architecture, and all models shared the same general input and output architecture. Each model began with a CNN stem that embedded the input image into a dense feature vector. Each model ended with a coordinate prediction head that predicted the (x,y,z) coordinates corresponding to the location where the image was captured. The experiments differed only in the architecture connecting the image embedding and the coordinate prediction head.

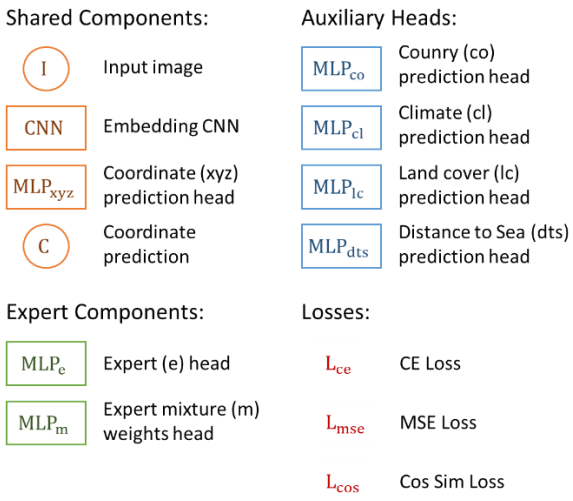


Figure 4: The components used to construct each architecture (see 4.2). The architecture of the CNN and each MLP is detailed in 4.3.

4.2 Experiment Architectures

Control: The first experiment served as the control condition. In this model, the image embedding was passed directly to the coordinate prediction head without any auxiliary prediction tasks or intermediate supervision.

Figure 5: Visualization of the control architecture.

Multi-loss: This model used the image embedding to predict both the final coordinates and several auxiliary labels. These auxiliary predictions contributed to the overall training loss. However, the auxiliary predictions were not used as inputs to the coordinate prediction head.

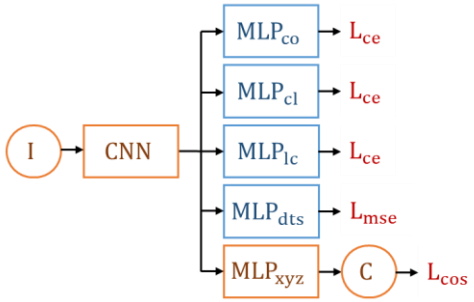


Figure 6: Visualization of the multi-loss architecture.

Parallel Auxiliary Heads: In this architecture, the auxiliary labels were predicted in parallel from the shared feature vector. The resulting auxiliary predictions were then concatenated with the original image embedding and passed to the coordinate prediction head.

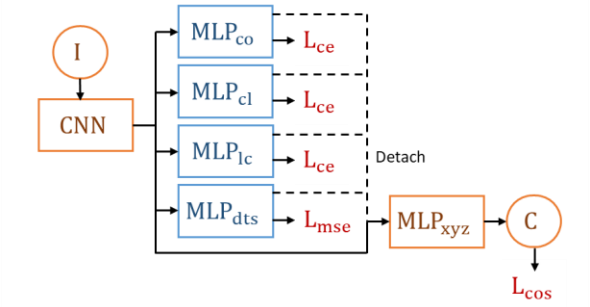


Figure 7: Visualization of the parallel heads architecture.

Sequential Auxiliary Heads: This model predicted auxiliary labels in a sequence from coarse to fine levels of information, and each predicted auxiliary representation was passed forward to the next stage. The output of this sequence of auxiliary prediction stages was passed to the coordinate prediction head.

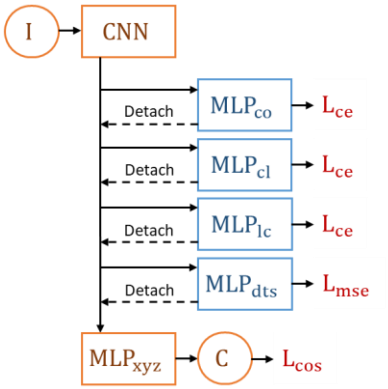


Figure 8: Visualization of the sequential heads architecture.

Mixture of Experts (MOE): In this model, the image embedding was used both to train multiple expert predictors and to make auxiliary predictions. The predicted auxiliary labels were then used to determine the weighting of the different experts. Finally, the weighted mixture of experts was concatenated with the auxiliary predictions, and this was passed to the coordinate prediction head.

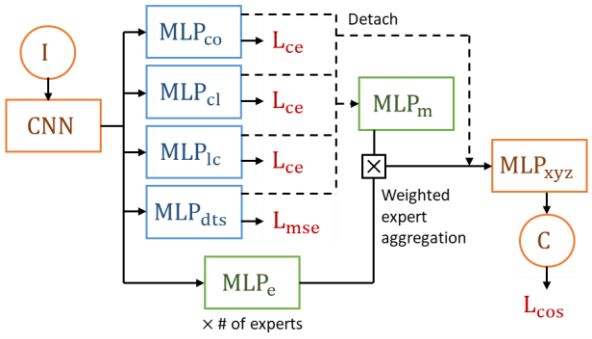


Figure 9: Visualization of the MOE architecture.

4.3 Model Components

All experimental architectures were constructed from a shared set of model components. This design ensured that differences in performance could be attributed to the way auxiliary predictions were incorporated, rather than to unrelated changes in the architecture. The three main components were the CNN stem, the feature prediction MPLs, and expert MLPs.

CNN Stem: all models used the same CNN stem to transform the input image into a dense feature embedding. The input to the stem was an RGB

image, and the output was a 128-dimensional feature vector.

The CNN consisted of a sequence of residual convolutional blocks. Each block first projected the input feature map to a new channel dimension using a 3×3 convolution, followed by batch normalization and a ReLU activation. A second 3×3 convolution and batch normalization layer were then applied to the projected feature map. The result was added back to the projected representation through a residual connection, and a final ReLU activation was applied.

The stem used seven convolutional blocks in total. The first block mapped the input image from 3 channels to 16 channels. Subsequent blocks increased the channel dimension from 16 to 32, then to 64, and finally to 128. The later blocks maintained 128 channels. Max pooling was applied after the first six convolutional blocks to reduce the spatial resolution of the feature maps. After the final convolutional block, the representation was passed through a convolutional head consisting of a 3×3 convolution, batch normalization, ReLU activation, and adaptive average pooling.

Feature Prediction MLPs: feature prediction MLPs were used to map learned feature vectors to either final coordinate predictions or auxiliary label predictions. Although the input and output dimensions varied depending on the role of the module, all feature prediction MLPs followed the same general three-layer structure.

Given an input feature vector of dimension d, the first linear layer projected the vector from d dimensions back to d dimensions, then performed batch normalization, a ReLU activation, and dropout. The second linear layer reduced the representation from d dimensions to d/2 dimensions, again followed by batch normalization, ReLU activation, and dropout. The final linear layer mapped the d/2 dimensional hidden representation to the desired output dimension.

Expert MLPs: the MOE model used expert MLPs to learn multiple specialized transformations of the shared image embedding. The expert mixture weight MLP was identical to the feature prediction MLPs. The expert MLPs were identical to the feature prediction MLPs, but without dimensionality compression.

4.4 Loss Functions

Several loss functions were used across the experiments to support both the primary coordinate prediction task and the auxiliary prediction tasks.

Coordinate Prediction Loss: the primary task in all experiments was to predict the geographic location associated with an input image. We chose to represent coordinates as three-dimensional unit-vectors rather than pairs of latitudes and longitudes to avoid the discontinuity introduced by longitude wraparound. As such, the models predicted a three-dimensional (x,y,z) vector, which was then normalized to unit length. The target latitude and longitude values were converted into this same three-dimensional unit-vector representation. The coordinate prediction loss was computed using cosine similarity between the predicted unit vector and the target unit vector. The loss was then defined as one minus the cosine similarity.

Haversine Loss: during evaluation, predictions were converted back into latitude and longitude and assessed using haversine distance in kilometers for interpretability.

Auxiliary Categorical Label Loss: several auxiliary tasks predicted categorical labels, including country, climate, and land cover. Each auxiliary head output logits over its label classes and was trained with class-weighted cross-entropy loss to account for imbalanced training labels. For each task, class weights were set proportional to the inverse frequency of each class in the training set and normalized to have a mean of one.

Auxiliary Continuous Label Loss: one auxiliary task, distance-to-sea, predicted a continuous label. The loss for distance-to-sea prediction was MSE between the predicted value and the target value.

4.5 Hyperparameters

Due to compute constraints, we could not perform an extensive hyperparameter search. Below is a description of hyperparameters we used, a small set of which we tested multiple values for.

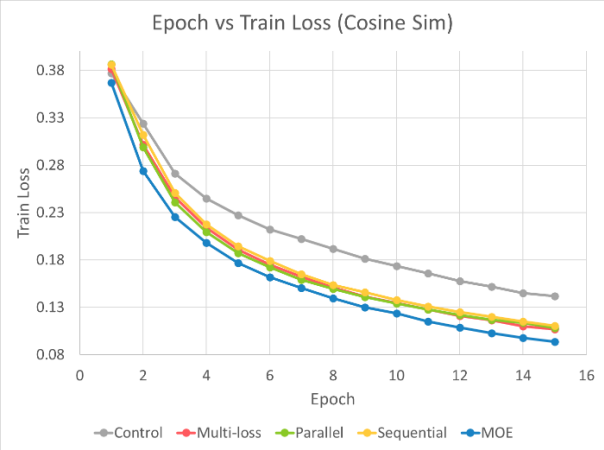
Across all experiments, we tested learning rates of 2e-3 and 1e-3. All models were trained using the Adam optimizer with a weight decay of 1e-4. The batch size was set to 32 and the embedding dimension was set to 128. Larger values of either resulted in CUDA out-of-memory errors. For each model, we tested dropout rates of 0.1 and 0.2. For the mixture-of-experts model, we additionally tested a dropout rate of 0.25 because this architecture contained more MLP modules. Each model trained for 15 epochs.

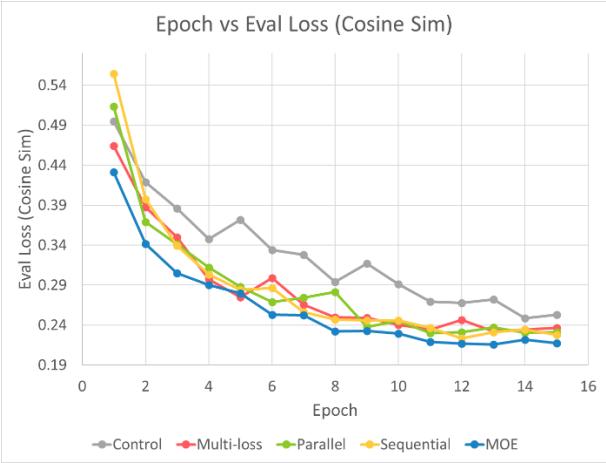
For all experiments except the control model, the total training objective combined the coordinate prediction loss with the auxiliary prediction losses. Since coordinate prediction was the primary task, the coordinate loss weight was set to 1.0, while each auxiliary loss was assigned a weight of 0.25.

The MOE model introduced several additional hyperparameters. The number of experts was set to 8 and the gate temperature was set to 1.0. To prevent expert collapse, an expert diversity loss weight was set to 0.02. To encourage the gating mechanism to distribute weight across multiple experts, an expert load-balance loss weight was set to 0.02.

5. Results and Analysis

5.1 Results





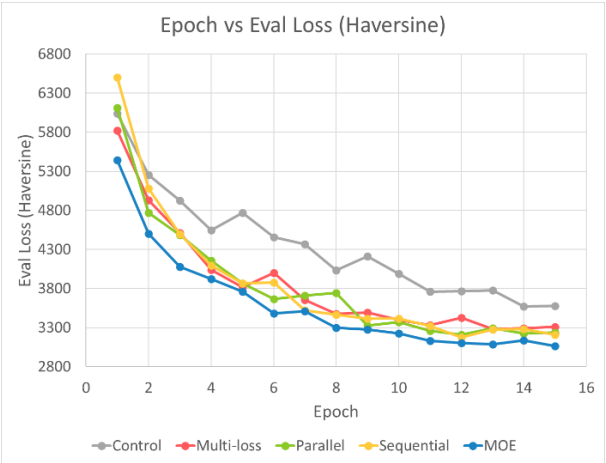


Figure 10: Top: Training loss over epochs for the five architectures. Middle: Eval cosine similarity loss over epochs for the five architectures. Bottom: Eval Haversine loss over epochs for the five architectures. Y-axis is in kilometers.

Lowest Losses across all Models

	Train (cs)	Eval (cs)	Eval (h)	Test (cs)
Control	0.1416	0.2486	3570.28	0.2481
Multi-loss	0.107	0.2322	3282.62	0.2312
Parallel	0.1082	0.23	3210.74	0.2258
Sequential	0.1106	0.2233	3174.35	0.2245
MOE	0.0937	0.2156	3061.75	0.2184

Table 1: Lowest train/eval/test losses for each model across all epochs. cs = cos sim loss. h = haversine loss.

Percent Reduction in Loss over Control

	Train (cs)	Eval (cs)	Eval (h)	Test (cs)
Multi-loss	24.44	6.6	8.06	6.81
Parallel	23.59	7.48	10.07	8.99
Sequential	21.9	10.18	11.09	9.51
MOE	33.83	13.27	14.24	11.97

Table 2: Percent reduction in loss relative to the control model for each architecture across train/eval/test. All values are percentages. cs = cos sim loss. h = haversine loss.

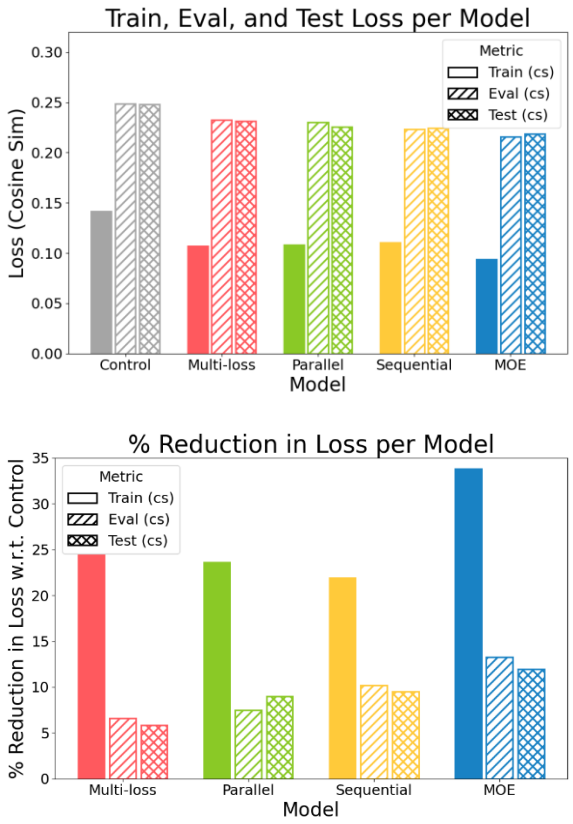


Figure 11: Top: Lowest train/eval/test losses for each model across all epochs. Bottom: Percent reduction in loss relative to the control model for each architecture across train/eval/test. All values are percentages. cs = cos sim loss.

5.2 Analysis

Performance: the control model had the highest training loss at every epoch, while the MOE model had the lowest training loss at every epoch. The MOE model's minimum training loss was 32.7% lower than the control model's minimum training loss. The multi-loss, parallel, and sequential auxiliary models remained close to one another, with training losses in between the control and MOE models.

A similar pattern appeared in evaluation. The control model had the highest cosine-similarity validation loss in 13 of the 14 epochs. The only exception was epoch 1, when the sequential and parallel models performed worse. The MOE model achieved the lowest cosine-similarity validation loss in 13 of the 14 epochs, with the only exception at epoch 5, when the multi-loss model performed best. Haversine validation loss followed the same general pattern. Relative to the control model's best cosine-similarity and haversine validation results, the MOE model reduced the lowest losses by 13.3% and 13.6%, respectively.

Validation and test performance were nearly identical across models. This is expected given the limited hyperparameter search, meaning the models were not extensively optimized on the validation set. As a result, the test set functioned less as a measure of performance after validation tuning and more as a second validation set.

Convergence: training loss did not fully plateau for any model within the training period. However, for all models except the control, cosine-similarity validation loss began approaching its minimum around epoch 8. By contrast, the control model continued to improve gradually through epoch 15, suggesting slower convergence.

The MOE model began with substantially lower loss than the other models and reached its minimum more quickly. The multi-loss, parallel, and sequential models converged at similar rates. The haversine validation loss followed the same convergence pattern as the cosine-similarity validation loss.

Smoothness: training loss decreased smoothly and monotonically for all models. Validation loss was less smooth, particularly for the control model. The control model showed a repeating pattern of validation-loss peaks at epochs 5, 9, and 13, with decreases between these peaks. The multi-loss model was the next least smooth, with validation loss peaks at epochs 6 and 12. Among the remaining models, the MOE model had the smoothest validation curve. The haversine validation loss showed the same overall smoothness pattern as the cosine-similarity validation loss.

Gap Between Training and Evaluation: across models, the lowest cosine-similarity validation losses were approximately 2.0 times higher than the lowest training losses. The control model had the smallest train–validation gap, with validation loss approximately 1.7 times higher than training loss. The MOE model had the largest gap, with validation loss approximately 2.2 times higher than training loss. This pattern is consistent with model capacity since the control model had the fewest parameters, while the MOE model had the most.

However, the gap does not necessarily indicate severe overfitting. Validation losses plateaued rather than increasing, and all models performed slightly better with a dropout rate of 0.1 than with 0.2. If overfitting were the dominant issue, stronger dropout might be expected to improve validation performance. Instead, these results suggest that the models may have been limited more by data coverage than by overfitting alone. In particular, the geographic sparsity and imbalance of the training data likely made generalization difficult even when the models continued to learn useful patterns from the training set.

6. Discussion

6.1 Interpretation

The control model performed worst, converged slowest, and produced the least stable validation curves. The MOE model performed best, converged fastest, and produced the most stable validation curves. These results suggest that auxiliary prediction tasks provided useful additional supervision. However, all auxiliary architectures were unable to break through a similar performance ceiling. This suggests that performance was constrained by the scale of the architecture and the amount of training data. Additionally, it is worth considering the advantages and limitations of each model.

The multi-loss model had the advantage of using auxiliary labels as additional supervision without increasing the complexity of the final coordinate prediction. However, because the auxiliary predictions were not passed into the coordinate prediction head, the model could only benefit from them indirectly through the shared image embedding. As a result, any geographic information captured by the auxiliary heads could not be used during evaluation.

The parallel auxiliary-head model addressed this limitation by concatenating the predicted auxiliary labels with the original image embedding before coordinate prediction. This gave the coordinate head access to intermediate geographic predictions. However, since the auxiliary predictions were detached before being passed forward, if an auxiliary prediction was inaccurate, the coordinate head had to learn around that error.

The sequential auxiliary-head model extended this idea by organizing auxiliary predictions from coarse to fine information. This enabled subsequent stages to have access to information from prior ones. However, it magnified the weakness that errors made early would propagate to later stages.

The MOE model outperformed all other architectures, suggesting that allowing different experts to specialize in different feature patterns and weighing those experts using auxiliary predictions improved final prediction. However, the MOE architecture introduced more parameters and hyperparameters. Given the limited hyperparameter search, the model may not have been able to fully benefit from its additional flexibility.

6.2 Limitations

The primary limitation of this project was compute, which affected model development in several ways.

For example, the image feature embedding was limited to 128 dimensions because larger embeddings caused CUDA out-of-memory errors. Although reducing the batch size or increasing checkpoints would have allowed larger embeddings, this would have substantially

increased training time. Since all auxiliary and coordinate predictions depended on the image embedding, this created an early bottleneck.

Additionally, compute limitations restricted model depth. The largest model used contained 14 convolutional layers and 27 linear layers (many of which were in parallel), which is relatively shallow compared with modern deep learning architectures. However, given the allotted compute, each model required approximately one day to train, so it was infeasible to further increase the scale. The same constraints limited hyperparameter exploration.

Compute limitations also restricted the amount of training data. The curated training set contained approximately 170,000 images. Although this is a substantial number of samples, it is sparse relative to roughly 148 million square kilometers of land on Earth. The OSV 5M dataset contained 25 times more data than could be used. However, doing so would have increased training time beyond the feasible scope of the project.

A separate limitation was the uneven geographic distribution of the dataset. For example, after filtering, South Africa was the only African country with enough samples to remain in the dataset, whereas 22 European countries passed the same threshold. As a result, Germany alone had approximately five times as many training samples as all of Africa. This imbalance likely caused the model to perform better on images from well-represented regions. This reflects a real-world bias in geographic data availability and representation.

7. Conclusion

Overall, models incorporating auxiliary tasks performed substantially better during training and showed slight improvements during evaluation and testing. However, more rigorous experimentation is needed before making stronger claims about the efficacy of these architectures. To this end, an immediate next step would be to scale both the amount of training data and the embedding dimension. Increasing the training dataset would also make it feasible to evaluate higher-capacity models without the concern of overfitting, such as architectures utilizing transformer blocks or additional experts. Future experiments could also investigate training schedules that warm up auxiliary heads before their predictions are passed into subsequent prediction stages.

8. Works Cited

[1] Haas, L., Skreta, M., Alberti, S., & Finn, C. (2024). PIGEON: Predicting image geolocations. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 12893–12902).

[2] He, K., Dong, T., Wu, J., & Zhang, J. (2024). One-step structure prediction and screening for protein-ligand complexes using multi-task geometric deep learning. arXiv. https://doi.org/10.48550/arXiv.2408.11356

[3] Hu, F., Jiang, J., Wang, D., Zhu, M., & Yin, P. (2021). Multi-PLI: Interpretable multi-task deep learning model for unifying protein–ligand interaction datasets. Journal of Cheminformatics, 13, Article 30. https://doi.org/10.1186/s13321-021-00510-6

[4] Müller-Budack, E., Pustu-Iren, K., & Ewerth, R. (2018). Geolocation estimation of photos using a hierarchical model and scene classification. In V. Ferrari, M. Hebert, C. Sminchisescu, & Y. Weiss (Eds.), Computer Vision – ECCV 2018 (pp. 575–592). Springer. https://doi.org/10.1007/978-3-030-01258-8_35

[5] Yan, J., Ye, Z., Yang, Z., Lu, C., Zhang, S., Liu, Q., & Qiu, J. (2024). Multi-task bioassay pre-training for protein-ligand binding affinity prediction. Briefings in Bioinformatics, 25(1), bbad451. https://doi.org/10.1093/bib/bbad451